Fast Flow Engine

FFENGINE

Community: *Low Code Airflow Native Data Integrator*

Enterprise: *Hybrid Queue-Centric Data Integration Engine*

Contents

1 Özet **7**

2 Fonksiyonel Analiz Çerçevesi **7**

2.1 FFEngine Community — Fonksiyonel Analiz 7

2.2 FFEngine Enterprise — Fonksiyonel Analiz 7

2.3 Sürüm Ayrımı İlkesi 8

3 Ürün Vizyonu **8**

4 Stratejik Amaç **8**

5 Problem Tanımı **8**

6 Amaç ve Kapsam **9**

6.1 Amaç 9

6.2 Kapsam 10

6.3 Proje Kapsamı Dışı 11

7 Hedef Kitle **12**

8 Pazarlama Bölümü **13**

8.1 7.1 Konumlandırma ve Değer Önerisi 13

8.1.1 1. Ürün Konumlandırma (Market Positioning) 13

8.1.2 2. Temel Değer Önerileri (Core Value Propositions) 13

8.1.3 3. Paydaş Bazlı Değer Matrisi 14

8.1.4 4. Sürüm Stratejisi: Toplulukla Büyü, Kurumsal Hızla Kazan 14

8.2 Ürün Katmanları: Community vs. Enterprise 14

8.3 Rekabet Avantajı (Neden ffengine?) . . . 15

8.4 Kritik Pazarlama Mesajları (Elevator Pitch) . . . 15

9 Community Mimari Yaklaşım **15**

9.1 Bileşenler . . . 15

10 Enterprise Mimari Yaklaşım **15**

10.1 Bileşenler . . . 15

10.2 Veri Akışı . . . 16

11 Mimari Genel Bakış (Dual-Core Architecture) **16**

11.1 ffengine Community (Python Core) . . . 16

11.2 ffengine enterprise . . . 16

12 Teknik Bileşen Detayları **17**

13 Teknik Mimari ve Bileşenler **17**

13.1 Veri Akış Protokolü (Streaming I/O) . . . 17

13.2 Dialect-Tabanlı Soyutlama (Agnostic Layer) . . . 17

13.3 Teknik Mimari Dokümanı . . . 18

13.3.1 Mimari Tasarım Felsefesi . . . 18

13.3.2 Çift Çekirdekli Ürün Yapısı (Dual-Core Architecture) . . . 18

13.3.3 Katmanlı Mimari Yapısı . . . 18

13.3.4 Veri Akışı ve Protokoller- Enterpise Versiyon . . . 19

13.3.5 Airflow Entegrasyonu ve Low-Code Yaklaşımı . . . 19

13.3.6 Airflow bu mimaride neden güçlü? . . . 20

13.3.7 Görsel Dönüşüm Arayüzü (Visual Mapper) . . . 20

13.3.8 ETL YAML ve DAG Oluşturma Ekranı . . . 20
13.3.9 ETL Aktarımlarında kullanılabilecek Mapping dosya yapısı . . . 20
13.3.10 Performans Hedefleri (KPIs) . . . 22
13.4 Teknik Gereksinimler ve Mimari Detaylar . . . 22
13.4.1 Temel İlke . . . 22
13.4.2 Ürün Diline Yansıması . . . 23
13.4.3 Ürün İçinde Modelleme: Üç Katmanlı Resolver . . . 23
13.4.4 Karar Kriterleri (Sadece DBMS Adına Bağlı Olmamalı) . . . 23
13.4.5 Capability Registry (Adapter Değil, Yetenek Kayıt Katmanı) . . . 24
13.4.6 Strategy Score Mekanizması . . . 24
13.4.7 Hard Fallback Zinciri . . . 25
13.4.8 Neden Kritik? . . . 25
13.4.9 Dialect-Tabanlı Soyutlama Katmanı (Database Agnosticism) . . . 25
13.4.10 Yüksek Performanslı Streaming I/O Mekanizması . . . 26
13.4.11 Bulk Yükleme Optimizasyonları (Target Specific) . . . 26
13.4.12 Konfigürasyon Yönetimi (Low-Code ETL) . . . 26
13.5 ETL ve Streaming Mimari Mimarisi (Deep Dive) . . . 27
13.5.1 ETL & Streaming Mimarisi (High-Speed Batch) . . . 27
13.5.2 Veri Akış Mimarisi (Data Flow Architecture) . . . 27
13.5.3 Teknik Bileşenler ve Protokoller . . . 28
13.5.4 Mimarinin Bileşen Şeması . . . 29
13.6 Teknik Ekibin yukarıdakilere ek olarak Uygulama Notları . . . 29

14 Airflow Entegrasyonu ve Operasyon **29**

15 Geliştirme Stack'i ve Standartlar **29**

16 Teknik Uygulama Detayları (Engineering Implementation) **30**

16.1 Veri Akış Protokolü (The Streaming Protocol) 30

16.2 Dialect Soyutlama Katmanı Uygulaması 30

16.3 Kullanılan Teknolojiler 30

16.4 Altyapı ve Dağıtım (DevOps & Infrastructure) 30

16.5 Hata Yönetimi ve Gözlemlenebilirlik (Observability) 31

17 İleri Düzey Teknik Detaylar (Advanced Technical Specs) **31**

17.1 Veri Tipi Uyumlaştırma ve Edge Case Yönetimi 31

17.2 Paralelizasyon Stratejisi 32

17.3 İzlenebilirlik (Logging & Telemetry) 32

18 Teknik Karar Kaydı (ADR — Architecture Decision Records) **32**

19 Gereksinimler **32**

19.1 Fonksiyonel Olmayan 32

20 Başarı Ölçütleri **32**

21 Pazar Araştırması: Pazar Konumlandırması ve Hedef Pazar Payı **33**

22 Riskler ve Varsayımlar **34**

23 Yol Haritası **34**

23.1 Kısa Vadeli (0–3 ay) 34

23.2 Orta Vadeli (3–6 ay) 34

23.3 Uzun Vadeli (6+ ay) 34

24 Stratejik Yol Haritası (2026–2027) **34**

24.1 Faz 1: Core & Community Launch (Q1 2026) 34

24.2 Faz 2: ffengine enterprise Core Development (Q1 2026) 35

24.3 Faz 3: Performance & Scalability (Q2 2026) 35

24.4 Faz 3: Market Expansion (2027) 35

25 FFEngine Feature Matrix (Community vs Enterprise) **35**

25.1 1) Execution Model & Runtime 35

25.2 2) Source (Okuma) 36

25.3 3) Target (Yazma) 36

25.4 4) Delivery & Reliability (Teslimat Semantiği) 36

25.5 5) Partitioning & Parallelism 37

25.6 6) Transformation 37

25.7 7) Configuration & Orchestration 37

25.8 8) Monitoring & Observability 37

25.9 9) Performance 38

25.10 10) DBMS Capability Kullanımı 38

25.11 Net Ürün Ayrımı (Tek Bakışta) 38

25.12 En Kritik Fark (Ürün Mesajı) 38

25.13 Enterprise Execution Strategy 38

25.13.1 Execution Model 39

25.13.2 Capability-Driven Strategy Selection 39

25.13.3 Queue-Aware vs Query-Centric 39

26 Kapsam Dışı Konular **39**

1 Özet

ffengine; finansal sistemler, bankacılık ve kurumsal veri ambarları için tasarlanmış, veritabanı bağımsız (database-agnostic) bir veri entegrasyon platformudur. PostgreSQL, MSSQL, Oracle gibi rdms sistemler veya Kafka gibi steaming tollarıyla, arasında çift yönlü veri aktarımını; kod yazımını minimize eden YAML tabanlı "ETL Studio" arayüzü ile Airflow aracılığıyla gerçekleştirir. Ürün, esnek bir Python katmanı sunan ffengine community ve "Hybrid Queue-Centric Dual-Core ETL Platform" olarak konumlanan, yüksek performanslı native teknolojiler barındıran ffengine enterprise olmak üzere iki temel çekirdekten oluşur.

2 Fonksiyonel Analiz Çerçevesi

Bu doküman, FFEngine ürün ailesinin iki farklı sürümü için fonksiyonel analizi birlikte sunar: **FFEngine Community** ve **FFEngine Enterprise**. Analiz, ortak platform kabiliyetlerini korurken her sürümün farklı çalışma modelini ve operasyonel hedeflerini ayrı başlıklar altında değerlendirir.

2.1 FFEngine Community — Fonksiyonel Analiz

Community sürümü için fonksiyonel kapsam aşağıdaki başlıklarda ele alınır:

- Low-code ETL Studio ile YAML tabanlı pipeline tanımı
- Airflow native orkestrasyon ve otomatik DAG üretimi
- Query-centric veri aktarımı (`fetchmany + executemany`)
- Standart kurumsal ETL ihtiyaçları için açık kaynak, genişletilebilir mimari

2.2 FFEngine Enterprise — Fonksiyonel Analiz

Enterprise sürümü için fonksiyonel kapsam aşağıdaki başlıklarda ele alınır:

- Queue-aware controlled data movement yaklaşımı
- Capability-driven strategy selection (kaynak/hedefe göre en uygun yürütme)
- Native runtime ile yüksek throughput, düşük gecikme ve düşük bellek izi
- Checkpoint, idempotency ve politika tabanlı teslimat semantiği

2.3 Sürüm Ayrımı İlkesi

Fonksiyonel analiz yaklaşımı tek bir yöntemi dayatmaz. Community tarafı ağırlıklı olarak query-centric akışı benimserken, Enterprise tarafı varsayılan olarak queue-aware modelde çalışır; ancak strateji seçimi tüm durumlarda kaynak ve hedef sistemlerin teknik yeteneklerine göre uyarlanır.

3 Ürün Vizyonu

ffengine, heterojen veritabanı ekosistemlerinde Apache Airflow'un orkestrasyon gücünü, yüksek performanslı low-code mimarisi ve ffengine enterprise çekirdeği ile birleştirerek veri aktarımını standardize eden; queue-aware ve controlled delivery yaklaşımıyla yüksek hız ile operasyonel kontrolü birlikte sunan bir ETL motoru olmayı hedefler.

Bu kapsamda hedef, Airflow'u bir streaming araca dönüştürmek değildir. Yaklaşım; uzun süren ETL aktarımlarını hem kaynakta hem hedefte küçük boyutlu veri pipe-line'larına bölerek, kontrollü mikro-batch akışlarıyla aktarım süresini kısaltmak ve operasyonel riski azaltmaktır.

4 Stratejik Amaç

Projenin ana amacı, geleneksel ve hantal batch tabanlı ETL sistemlerinden modern, Pythone bazlı kullanımı kolay veri aktarım aracına dönüştürürken, enterpise versiyonda streaming mimarisine geçiş yaparak veri işleme performansını ve gerçek zamanlı entegrasyon kabiliyetini artırmaktır. ffengine ile hedeflenen stratejik kazanımlar şunlardır:

- Hız ve Verimlilik: Veri geliştirme ve pipeline kurulum süresini "saatlerden dakikalara" indirmek.
- UI: ETL geliştirmelerini UI yardımıyla yapılabilmesi, Veri müjendidleri dışındaki ekiplerinde aktarım yapabilmesinin sağlanması
- Airflow native: Airflow içersinden çalışan yapısıyla başka toollara ihtiyaç duyulmaması.
- Esneklik: Dialect tabanlı soyutlama ile tek bir çekirdek kod üzerinden farklı RDBMS sistemlerini (MSSQL, Oracle, Postgres, vb) yönetebilmek.
- Maliyet Tasarrufu: Bakım maliyetlerini düşürmek ve yeni veritabanı kaynaklarının eklenmesini kolaylaştırarak operasyonel çeviklik sağlamak.

5 Problem Tanımı

Kurumsal veri ekosistemlerinde yüksek hacimli ETL akışları, aşağıdaki teknik problemler nedeniyle sürdürülebilir performans sağlayamaz:

- Airflow: Airfow kullanımında python yazılım ihtiyacı
- Kodlama: ETL kodlamasının her geliştirici için farklı tarzda yapılması
- Darboğazlar: JSON/CSV ara formatlarına yazma ve okuma adımları, I/O ve CPU üzerinde ciddi bottleneck yaratır.
- OOM Hataları: Büyük tabloların tümünün belleğe alınması, bellek taşmalarına ve worker çökmesine yol açar. Enterpirse versiyonda düşük bellekli çoklu pipline yapsı.
- Yükleme Süreleri: Milyon satırlık veri transferlerinde batch tabanlı yöntemler saatler süren gecikmeler oluşturur.
- Heterojen Sistemler: Oracle, MSSQL ve Postgres gibi farklı RDBMS'ler arasında tutarlı ve hızlı veri akışı kurarken, tüm yük ETL serverda olması
- Lisans: Data Integration yazılım lisans maliyetlerinin yüksekliği

6 Amaç ve Kapsam

6.1 Amaç

ffengine (Fast Flow Engine) projesinin Community versyionda temel amacı; Airflow'a ekleen UI ile uygulamayı Low-code, Airflow native bir ETL aracına dönüştürmek. Veri aktarımında, kodların standartlaşmasını sağlamak. Aktarımları veri mühendisleri dışında analistlerinde yapabilmesini sağlamak. Aktarımları yaparken ffengine'nin partition özellikleriyle birlikte Airflow'un concurrent özelliklerini birleştirmek. Open source community ile ürüne yeni özellikler ekleme. ffengine (Fast Flow Engine) projesinin Entepise versyionda temel amacı; geleneksel batch tabanlı ETL (Extract-Transform-Load) sistemlerinden, yüksek performanslı bir streaming (akış) mimarisine geçiş yaparak. Comunity versiyondaki özelliklere ek olarak Python engine yerine C' ye dönüştürülmüş streaming mimaride engine ile veri işleme performansını maksimize etmek. Projenin stratejik hedefleri şu ana başlıklar altında toplanmaktadır:

1. Community ve Enterprise Versiyonları
    - Community versiyonu ile; UI kullanımı, oromatik DAG ve YAML oluşturma özellikleri, Mapping adımları ile tam ve eksiksiz ETL deneyimi sağlanması amaçlanmaktadır.
    - Enterprise versiyon ile; veriyi streaming mimaride aktararak community versiyondaki hıza ek olarak hızlı veri işleme yetenekleri kazanmaktadır.
    - Enterprise versiyon ile; 7/24 servis desteği sağlanabilir. Lisans ve bakım gelir modeli.
    - Community versiyonda open source yapısıyla geliştirmye açıklık
2. Performans ve Ölçeklenebilirlik
    - Yüksek Hacimli Veri Yönetimi: Milyon satır ve üzerindeki yüklemelerde performans optimizasyonu sağlayarak, kurumsal veri ambarları için ölçeklenebilir bir altyapı oluşturmak.
    - Binary Streaming: Veriyi JSON tabanlı serileştirme maliyetine girmeden "binary streaming" yaklaşımıyla taşıyarak aktarım hızını artırmak.

- Bellek Verimliliği: Büyük tabloların belleği şişirmeden, server-side cursor yöntemleriyle kademeli okunmasını ve hedefe hızla yazılmasını sağlamak.
- ffengine enterprise Gücü: Kritik finansal sistemlerde küçük paketlerle kilitlenmeyi azaltan queue-aware akış tasarımı, producer/consumer tabanlı controlled delivery ve düşük seviyeli (low-level) çekirdek ile ultra yüksek veri geçiş hızı (throughput) elde etmek.

3. Veritabanı Bağımsızlığı (Database Agnosticism)

   - Çoklu RDBMS Desteği: Halihazırda PostgreSQL üzerinde çalışan yapıyı; MSSQL ve Oracle veritabanlarını da destekleyecek şekilde genişletmek.
   - Dialect-Temelli Soyutlama: Veritabanına özgü SQL lehçelerini ve veri tiplerini merkezi bir soyutlama katmanı (Dialect Layer) üzerinden yöneterek, çekirdek kodu veritabanı bağımsız hale getirmek.
   - Esnek Bağlantı: Kurumsal altyapılara tam uyum için ODBC ve JDBC bağlantı seçeneklerini devreye almak.
   - Kafka ve CDC desteği ile streamin veri kaynaklar ve hedefleri desteklemek

4. Operasyonel Hız ve Otomasyon (Low-Code ETL)

   - ETL Studio Deneyimi: Apache Airflow üzerinde çalışan kodsuz/düşük kodlu bir arayüz sunarak, veri ekiplerinin her tablo için ayrı kod yazma zorunluluğunu ortadan kaldırmak.
   - Geliştirme Süresinin Kısaltılması: Manuel pipeline geliştirme süreçlerini "saatlerden dakikalara" indirmek.
   - Otomatik Şema Yönetimi: Kaynak veritabanındaki metadata bilgilerini kullanarak hedef sistem için uygun DDL (CREATE TABLE) komutlarını ve veri tipi eşlemelerini otomatik üretmek.
   - Konfigürasyon Odaklılık: Tüm ETL görevlerini YAML dosyaları üzerinden, kod değişikliği gerektirmeden yönetilebilir kılmak.

Özetle ffengine; Airflow'un güçlü orkestrasyon kabiliyetini, ffengine enterprise çekirdeğinin ham hızıyla birleştirerek kurumsal sistemler arasındaki veri trafiğini bir mühendislik yükü olmaktan çıkarmayı hedefler.

6.2 Kapsam

Kurumsal veri ekosistemlerinde yüksek performanslı, veritabanı bağımsız (database-agnostic) ve ölçeklenebilir bir ETL altyapısı sunmaktır. Proje, geleneksel batch tabanlı veri işleme yöntemlerinden modern streaming (akış) mimarisine geçişi temel alır. Projenin kapsamı şu ana başlıklar altında detaylandırılmıştır:

1. Veritabanı ve Bağlantı Kapsamı

   - Çoklu RDBMS Desteği: Proje; PostgreSQL, MSSQL ve Oracle veritabanları arasında çift yönlü veri aktarımını sağlar.
   - Geniş Bağlantı Seçenekleri: Native DBAPI sürücülerinin yanı sıra kurumsal altyapılara uyum için ODBC ve JDBC bağlantı seçeneklerini içerir.

2. Mimari ve Fonksiyonel Kapsam

- Dialect-Tabanlı Soyutlama: Veritabanına özgü SQL sentaksları (quoting, pagination, bulk insert) ve veri tiplerini merkezi bir katman üzerinden yönetir.
- Otomasyon Motoru: Kaynak veritabanından metadata okuyarak hedef sisteme uygun DDL (CREATE TABLE) üretimi ayrıca veri tipi eşlemesini (Type Mapping) otomatikleştirir.
- Dinamik Parçalama (Partitioning): Farklı veritabanları için optimize edilmiş auto, percentile, hash_mod ve explicit gibi çeşitli parçalama stratejilerini kapsar.

3. Ürün Katmanları (Edition Scope)
   - ffengine community: Python tabanlı, standart ETL süreçlerini Airflow operatörü üzerinden yöneten açık kaynaklı çekirdeği kapsar.
   - ffengine enterprise: Düşük seviyeli (low-level) C/C++ çekirdeği kullanarak "Zero-Copy Streaming" ile ultra yüksek performanslı veri taşıma kabiliyetini kapsar.
4. Orkestrasyon ve Kullanıcı Deneyimi
   - ETL Studio: Apache Airflow arayüzüne entegre, kod yazmadan (low-code) YAML konfigürasyonu ile pipeline tanımlamaya olanak tanıyan bir "ETL Studio" deneyimi sunar.
   - Auto-DAG Generator: Konfigürasyon dosyalarından otomatik olarak Airflow DAG'leri üreterek geliştirme süresini saatlerden dakikalara indirir.
5. Performans ve Operasyonel Hedefler
   - Bellek ve Hız: Milyon satırlık verinin MB seviyesinde RAM kullanımıyla pipline'lar aracılığıyla kısa sürede aktarılması projenin performans sınırlarını belirler.
   - Ölçeklenebilirlik: Airflow'un "Dynamic Task Mapping" özelliğini kullanarak veri hacmine göre otomatik worker ölçeklemesi yapabilmeyi kapsar.

6.3 Proje Kapsamı Dışı

Bu proje kapsamı dışındaki konular;

- Veri Kalitesi ve Validasyon (Data Quality): Akış anında veri tiplerinin doğruluğu dışında, iş kurallarına dayalı (ör. null kontrolü, regex eşleşmesi) kontrol mekanizması.
- Veri Maskeleme ve Anonimleştirme (Data Masking): Finansal verilerle çalışırken KVKK/GDPR uyumu için hassas kolonların akış anında maskelenmesi.
- Change Data Capture (CDC): ffengine enterprise bir CDC aracı değildir; veritabanı loglarından (WAL/Redo log) gerçek zamanlı değişim yakalama özelliği kapsam dışıdır. Enterprise katmanında Kafka veya RapidMQ kaynak ya da hedef sistem olarak kullanılabilir ve FFEngine bu akışlar arasında yüksek hızlı veri taşıma görevini üstlenir.

7 Hedef Kitle

ffengine, özellikle yüksek hacimli veri trafiğine sahip, performansın kritik olduğu ve karmaşık veritabanı ekosistemlerini yöneten kurumsal yapılar için tasarlanmıştır. Projenin birincil hedef kitlesini oluşturan kurum tarzları şunlardır:

1. Finans ve Bankacılık Kuruluşları
   - Günlük milyonlarca işlemin (transaction) gerçekleştiği ve bu verilerin hızlıca analitik ortamlara taşınması gereken finansal sistemler.
   - Yüksek güvenlik ve veri tutarlılığı gerektiren bankacılık altyapıları.
   - Eski nesil (legacy) sistemler ile modern veri ambarları arasında köprü kurmaya ihtiyaç duyan kurumlar.
2. Büyük Veri (Big Data) Odaklı Şirketler
   - Milyonlarca satır veriyi düzenli olarak işlemek zorunda olan, veri hacmi arttıkça performans kaybı yaşayan kurumlar.
   - Veri ambarı (Data Warehouse) ve veri gölü (Data Lake) mimarilerini yöneten, verinin tazeliğine (data freshness) önem veren yapılar.
   - Ölçeklenebilir ETL süreçlerine ihtiyaç duyan e-ticaret, perakende veya telekomünikasyon devleri.
3. Homojen veya Heterojen Veritabanı Kullanan Kurumlar
   - Altyapısında aynı anda Oracle, MSSQL ve PostgreSQL gibi farklı RDBMS sistemlerini barındıran ve bunlar arasında kesintisiz veri akışı hedefleyen kuruluşlar. Enterprise versiyonda streaming yapıları olan veya CDC kullanan kurumlar.
   - Veritabanı bağımsız (database-agnostic) bir mimari ile vendor-lock (tek bir sağlayıcıya bağımlılık) riskini azaltmak isteyen teknoloji ekipleri.
4. Mühendislik ve Teknoloji Ekipleri
   - Her tablo için ayrı kod yazmak yerine, Low-Code (düşük kodlu) ve konfigürasyon odaklı (YAML) sistemlerle operasyonel yükü azaltmak isteyen veri mühendisliği ekipleri.
   - Halihazırda Apache Airflow kullanan ve orkestrasyon gücünü yüksek hızlı bir veri taşıma motoruyla (ffengine enterprise) birleştirmek isteyen teknoloji odaklı şirketler.
   - Veri mühendisleri yanı sıra veri analistlerin veya benzer pozisyondaki çalışanlarında aktarım yapması gereken kurumlar.

Özetle ffengine; veriyi sadece "taşımak" değil, kurumsal ölçekte en hızlı ve en az maliyetli şekilde yönetmek isteyen, veri yoğunluğu yüksek tüm modern işletmeleri hedeflemektedir.

8 Pazarlama Bölümü

8.1 7.1 Konumlandırma ve Değer Önerisi

8.1.1 1. Ürün Konumlandırma (Market Positioning)

ffengine, geleneksel batch tabanlı hantal ETL araçları ile karmaşık CDC çözümleri arasındaki boşluğu dolduran, "High-Speed Transport" (Yüksek Hızlı Taşıma) odaklı bir katmandır.

- Pazar Konumu: Pazardaki çoğu araç (Airbyte, Fivetran vb.) Java tabanlıdır ve veriyi JSON olarak serileştirir; bu da yüksek hacimli verilerde darboğaz yaratır. ffengine Enterprise, kurumsal veriyi "Binary Streaming" protokolüyle, ham hızda taşıyarak bu darboğazı ortadan kaldırır.
- Tamamlayıcı Rol: dbt gibi "dönüşüm" (Transform) odaklı araçların rakibi değil, onları en hızlı veriyle besleyen "Lojistik Çekirdek"tir.

8.1.2 2. Temel Değer Önerileri (Core Value Propositions)

A. Mühendislik Ekonomisi: "Saatlerden Dakikalara"

- Veri mühendisliği eforunu manuel kod yazımından kurtararak operasyonel verimliliği maksimize eder.
- Low-Code/No-Code: Airflow üzerinde her tablo için ayrı DAG yazmak yerine, merkezi bir YAML konfigürasyonu ile pipeline kurulum süresini %80 oranında azaltır.
- Otomatik Şema Yönetimi: Kaynak metadata bilgilerini okuyup hedef DB'ye uygun DDL'i (CREATE TABLE) ve veri tipi eşlemelerini saniyeler içinde otomatik üretir.

B. Rakipsiz Performans ve Altyapı Tasarrufu

- ffengine enterprise çekirdeği sayesinde, donanım kaynaklarını en verimli şekilde kullanarak maliyetleri düşürür.
- Binary Fast Lane: JSON serileştirme yükünü bypass ederek veri aktarım hızını 10 kata kadar artırır.
- Düşük Kaynak Tüketimi: Milyon satırlık devasa veri setlerini, sabit RAM kullanımıyla belleği şişirmeden taşır.

- Zero-Copy Streaming: Enterpirse versiyonda, veriyi Python nesnelerine dönüştürmeden, doğrudan C seviyesindeki buffer'lar üzerinden kaynaktan hedefe pompalar.

C. Kurumsal Esneklik ve Güvenilirlik

- Karmaşık veritabanı ekosistemlerini tek bir standart altında birleştirir.
- Database-Agnostic: Tek bir çekirdek kod ile PostgreSQL, MSSQL, Oracle vb DBMS'ler arasında çift yönlü veri aktarımı sağlar.
- Orkestrasyon Gücü: Apache Airflow'un yerleşik ölçeklenme kabiliyetini kullanarak, veri hacmine göre worker ölçeklemesi yapar ve iş yükünü dinamik olarak dağıtır.

8.1.3 3. Paydaş Bazlı Değer Matrisi

Paydaş	**ffengine ile Kazanımı**
CTO / Veri Direktörü	Altyapı maliyetlerinde (Cloud/On-prem) tasarruf
Veri Mimarı	Vendor-lock riskini ortadan kaldıran, tüm DB'leri tek dilde (YAML) konuşturan standartlaşmış bir ETL mimarisi.
Veri Mühendisi	Tekrarlayan "boilerplate" kodlardan kurtulur

8.1.4 4. Sürüm Stratejisi: Toplulukla Büyü, Kurumsal Hızla Kazan

- ffengine community: Veri mühendisliği topluluğuna, Airflow üzerinde çalışan en esnek ve kodsuz ETL standartlarını ücretsiz sunarak pazarı domine eder.
- ffengine enterprise: Performansın "hayat memat" meselesi olduğu finansal ve devasa hacimli yapılar için C-tabanlı, ultra hızlı ve 7/24 destekli profesyonel çözümdür.

8.2 Ürün Katmanları: Community vs. Enterprise

Pazarlama ekibi, "ücretsiz" ve "ücretli" versiyon arasındaki farkı net bir performans çizgisiyle anlatmalıdır:

- ffengine community: Topluluk odaklı, Python tabanlı ve standart kurumsal ihtiyaçları Airflow orkestrasyonu ile çözen açık kaynaklı ETL çekirdeği.
- ffengine enterprise: Kritik finansal veriler ve devasa hacimler için geliştirilmiş, düşük seviyeli (low-level) C çekirdeği ile "Zero-Copy Streaming" yapan profesyonel versiyon.

8.3 Rekabet Avantajı (Neden ffengine?)

Kıyas	Açıklama
Geleneksel Araçlar (Batch)	Veriyi diske yazar, belleği şişirir ve yavaştır.
CDC Araçları	Kurulumu karmaşıktır ve her zaman gerekli değildir.
ffengine Farkı	Batch veriyi "Streaming" hızıyla taşır. Veriyi parçalamadan, "binary" bir kanal üzerinden doğrudan hedefe akıtır.

8.4 Kritik Pazarlama Mesajları (Elevator Pitch)

ffengine, sistemler arasında veri taşımayı bir mühendislik yükü olmaktan çıkarır. Airflow'un gücünü arkasına alan low-code ETL Studio ile kod yazmadan pipeline kurun; ffengine enterprise ile verinizi streaming ile taşıyın.

9 Community Mimari Yaklaşım

9.1 Bileşenler

- Extractor (Python Operator)
- Transformer/Mapper (Type Mapping & DDL)
- Loader (Python Operator)

10 Enterprise Mimari Yaklaşım

10.1 Bileşenler

- Extractor (Source Reader)
- Stream Engine (Buffer/Backpressure)
- Transformer/Mapper (Type Mapping & DDL)
- Stream Engine (Buffer/writter)
- Loader (Target Writter)

10.2 Veri Akışı

ETL akışı, "Extract → Stream → Load" sırası ile çalışır. Kaynaktan okunan veri, akış motoru üzerinden backpressure kontrolüyle hedef sisteme taşınır; gerekli tip dönüşümleri ve DDL üretimi bu akış sırasında uygulanır.

11 Mimari Genel Bakış (Dual-Core Architecture)

ffengine, geniş kitlelere ulaşmak için esnek bir Python katmanı (Community) ve en yüksek performans için düşük seviyeli (low-level) bir C çekirdeği (Enterprise) üzerine inşa edilmiştir.

11.1 ffengine Community (Python Core)

Amacı: Kolay kurulum, geniş veritabanı desteği ve standart ETL süreçleri.

- Dil: Python 3.10+.
- Bağlantı: Standart DBAPI 2.0 (Psycopg, cx_Oracle, pyodbc).
- Metod: `fetchmany` ve `executemany` tabanlı batch processing.
- Orkestrasyon: Apache Airflow üzerinde FFEngineOperator.
- Arayüz: YAML tabanlı ETL Studio konfigürasyonu.

11.2 ffengine enterprise

Amacı: Milyarlarca satırlık veride minimum CPU/RAM kullanımı, kontrollü teslimat (controlled delivery) ve maksimum throughput.

- Dil: Pure C / C++ (Python extension veya bağımsız binary).
- Bağlantı: Native OCI (Oracle), libpq (Postgres) ve SQL Server Native Client.
- Çekirdek Metod: Zero-Copy ve Binary Streaming. Veri, Python nesnesine dönüştürülmeden C seviyesindeki buffer'lar üzerinden kaynaktan hedefe aktarılır.
- Ingress Queue: Kaynaktan gelen akışların sıra kontrollü alınması için giriş kuyruğu katmanı.
- Egress Queue: Hedefe yazma öncesi kontrollü çıkış ve akış dengelemesi sağlayan çıkış kuyruğu katmanı.
- FFEnvelope: Her veri paketini metadata (kaynak, hedef, sıra, deneme sayısı, zaman damgası) ile taşıyan standart paketleme yapısı.

- Ack/Nack Mekanizması: Başarılı teslimatta ack, hata durumunda nack üreterek güvenli tekrar deneme akışı sağlar.
- Checkpoint: Uzun akışlarda kaldığı yerden devam edebilmek için offset/anahtar bazlı ilerleme kaydı.
- Delivery Policy: At-least-once, retry/backoff ve timeout politikalarını akış tipine göre yöneten teslimat kuralları.
- Adaptive Micro-Batch: Hedef sistem yüküne göre paket boyutunu dinamik ayarlayan mikro-batch yaklaşımı.
- Multi-Lane Pipeline: Aynı akışı birden fazla paralel lane'e bölerek kontrollü eşzamanlı işleme.
- Target Controlled Writer: Hedef sistemin yazma kapasitesi ve lock davranışına göre kontrollü yazma hızı/stratejisi seçimi.
- Optimizasyon: SIMD talimat setleri ve işlemci seviyesinde paralelizasyon.

12 Teknik Bileşen Detayları

13 Teknik Mimari ve Bileşenler

ffengine, "Batch" dünyasından "Streaming" dünyasına geçişin teknik manifestosu olarak kurgulanmıştır.

13.1 Veri Akış Protokolü (Streaming I/O)

- Piping Mantığı: Veri, ara depolama veya devasa bellek kullanımı olmadan doğrudan kaynaktan hedefe akıtılır.
- Binary Fast Lane: JSON serileştirme yükünü bypass ederek veriyi ikili (binary) formatta taşıyan "fast lane" yaklaşımı kullanılır.
- Backpressure Yönetimi: Hedef veritabanı yazma hızına göre kaynak okuma hızı otomatik dengelenir, RAM taşmaları önlenir.

13.2 Dialect-Tabanlı Soyutlama (Agnostic Layer)

- Evrensel Çekirdek: Tek bir kod tabanı üzerinden PostgreSQL, MSSQL ve Oracle desteği sağlanır.
- Otomatik Şema Yönetimi: Kaynak metadata okunarak hedef veritabanına uygun DDL (CREATE TABLE) komutları otomatik üretilir.

13.3 Teknik Mimari Dokümanı

13.3.1 Mimari Tasarım Felsefesi

ffengine, geleneksel "Veriyi oku → Diske/RAM'e yaz → Hedefe gönder" döngüsünü kırarak, veriyi bir boru hattı (pipeline) içinden akan kesintisiz bir su gibi yönetir. Sistem, batch veri transferini modern streaming I/O teknikleriyle gerçekleştirerek ara depolama veya devasa bellek kullanımına ihtiyaç duymaz.

13.3.2 Çift Çekirdekli Ürün Yapısı (Dual-Core Architecture)

Ürün, kullanım senaryosuna göre iki farklı motor seçeneği sunar:

- ffengine community (Python Core): Esnek Python katmanı üzerine inşa edilmiştir. Standart DBAPI 2.0 (Psycopg, cx_Oracle, pyodbc) bağlantılarını kullanarak `fetchmany` ve `executemany` tabanlı batch işleme yapar.
- ffengine enterprise: En yüksek performans için düşük seviyeli (low-level) C/C++ çekirdeği üzerine inşa edilmiştir. Native kütüphaneler (OCI, libpq vb.) üzerinden Zero-Copy Streaming yaparak veriyi Python nesnesine dönüştürmeden doğrudan C seviyesindeki buffer'lar üzerinden kaynaktan hedefe aktarır. Standart metodlara göre 5x–10x performans artışı sağlar.

13.3.3 Katmanlı Mimari Yapısı

Sistem, birbirine gevşek bağlı (decoupled) yedi ana katmandan oluşur:

- Orkestrasyon Katmanı (Apache Airflow): Görev planlama, izleme ve FFEngineOperator üzerinden motorun tetiklendiği katmandır.
- Airflow Orchestration ile Çoklu Paralel Çalıştırma: Dynamic Task Mapping ve concurrent task yapısıyla aynı anda birden fazla aktarım görevinin paralel olarak yürütülmesini destekler.
- ETL Studio Arayüz (Apache Airflow- UI): Uygulamanın Ariflow native olmasını sağlayan en önemli özelliklerinden biri olan, Airflow üzerine eklenen sayfa ile eni DAG ve YAML otamatik üretiliyor. Bu sistem ile oluşturulan DAG' lerde değişiklik yapılabiliyor.
- Yönetim Katmanı (ETLManager): Konfigürasyon okuma, strateji seçimi, logging ve metrik yönetimini gerçekleştirir. Community version ve Enterprise versiyon burada farklı çalışır.
- Soyutlama Katmanı (Dialect Layer): Veritabanına özgü SQL sentakslarını (quoting, pagination, bulk insert) kapsülleyerek çekirdek kodu veritabanı bağımsız (agnostic) hale getirir. Saf SQL sözdizimi: quoting, literal üretimi, predicate, LIMIT/OFFSET, literal_for_declared_type, _escape_sql_string, _as_date_literal vb.

- Sürücü Katmanı (Driver): Bağlantı + I/O: connect, fetchall, server_cursor_iter, copy_to_filelike, insert_dataframe, get_column_types vb. Dialect'e self.dialect üzerinden erişir. DBAPI, ODBC veya JDBC üzerinden veritabanıyla fiziksel bağlantıyı sağlar.
- Session Katmanı: Üst katman facade: factory'den driver seçimi, context manager (with), otomatik commit/close.
- Operasyon Katmanı (Extractor/Transformer/Loader): Server-side cursor ile veri çekme, "on-the-fly" dönüşüm(Tranform - ETL) ve hedef DB'ye özel bulk yazma işlemlerini yürütür. Community version ve Enterprise versiyon burada farklı çalışır.

13.3.4 Veri Akışı ve Protokoller- Enterpise Versiyon

Streaming I/O ve Binary Fast Lane
Geleneksel ETL araçları veriyi JSON veya CSV olarak geçici dosyalara yazarken, ffengine şu yolu izler:

- Binary Stream: Özellikle PostgreSQL tarafında psycopg3 kullanarak veriyi metin tabanlı değil, binary formatta aktarır; bu sayede CPU kullanımını psycopg2 göre %30–%40 azaltır.
- Backpressure (Geri Basınç) Yönetimi: Hedef veritabanı yazma hızı yavaşlarsa, Streamer modülü kaynak okuma hızını otomatik dengeler ve RAM taşmalarını önler. RAM max ayarları burada etkili olur.

Dialect-Aware SQL ve DDL Üretimi

- Dynamic DDL: Kaynak tablonun metadata bilgilerini (kolon ismi, tipi, nullable durumu) okuyarak hedef veritabanına uygun CREATE TABLE komutunu anlık üretir.
- Type Mapping: Veri tiplerini (ör. Oracle NUMBER vs PostgreSQL BIGINT) merkezi bir TypeMapper sınıfı üzerinden otomatik olarak eşler.
- Enterpise versiyonda streaming yapılamayan tüm aktarımlar, SQL aktarımı yada standart aktarım yöntemleriyle yapılmaya devam eder.

13.3.5 Airflow Entegrasyonu ve Low-Code Yaklaşımı

- ETL Studio: Airflow arayüzü içinde çalışan, sürükle-bırak veya YAML konfigürasyonu ile kod yazmadan pipeline tanımlama imkanı sunan eklentidir.
- Auto-DAG Generator: Tanımlanan YAML konfigürasyonlarından otomatik olarak Airflow DAG'leri üretir.
- Dynamic Task Mapping: Veri hacmine göre worker sayısını otomatik artırarak paralel yükleme (Horizontal Scaling) sağlar.

13.3.6 Airflow bu mimaride neden güçlü?

Airflow'un Enterprise versiyonda çok uygun gelen birkaç güçlü özelliği vardır:

- Dynamic Task Mapping: Çalışma zamanında veri/parça sayısına göre görevler çoğaltılabilir; böylece küçük paketler paralel lane'lere bölünebilir.
- Deferrable Operators: Bekleyen veya polling yapan task'ların worker slot işgal etmeden triggerer üzerinde beklemesini sağlar; bu da çok sayıda küçük queue akışı için kritik avantaj sunar.
- Airflow Pools: Aynı anda belirli bir sisteme kaç task yükleneceği, ne kadar bellek tüketilebileceğini sınırlandırarak kontrollü paralellik sağlar.

Bu yaklaşım, ürünün hedeflediği "küçük boyutlu birden fazla akışı aynı anda ama kontrollü çalıştırma" modeline doğrudan uyum sağlar.

13.3.7 Görsel Dönüşüm Arayüzü (Visual Mapper)

Kullanıcılar YAML'ı ve DAG'i , Airflow'a eklenen ETL Studio sayfası üzerinden görsel olarak oluşturabilecektir. Aşağıdaki ekran, "ETL Studio" giriş butonunu göstermektedir.

13.3.8 ETL YAML ve DAG Oluşturma Ekranı

ETL YAML dosyası konfigürasyonunun ve DAG dosyasının otomatik oluşmasını sağlayan sayfa aşağıda gösterilmektedir. Bu sayfa Airflow embedded olarak çalışmaktadır.

13.3.9 ETL Aktarımlarında kullanılabilecek Mapping dosya yapısı

Mapping yapısı, kaynak-hedef eşleşmesini dosya sistemi hiyerarşisi üzerinden standartlaştırır. Her görev için bağımsız bir mapping dosyası tutulur. Böylece sürümleme, denetim ve bakım süreçleri sadeleşir. Dosya isimlendirmesi kaynak şema, kaynak tablo, hedef şema ve hedef tablo bilgisini açık şekilde yansıtacak biçimde tanımlanır. Mapping dosyası üretmek için ayrı bir program kullanılacaktır.

Önerilen klasörleme yaklaşımı aşağıdadır:

```
mappings/
```

```
  source_schema/
    source_table_to_target_schema_target_table_v1.yaml
```

Örnek mapping dosyaları:

```yaml
version: '1.0'
mapping_type: column
generated_at: '2025-10-07T10:14:49Z'
source:
  schema: ocn_iss
  table: crd_account_balance_used_hist
columns:
- source: guid
  target: guid
  type: NUMERIC(16, 0)
- source: last_updated
  target: last_updated
  type: NUMERIC(16, 0)
- source: txn_guid
  target: txn_guid
  type: NUMERIC(16, 0)
- source: txn_def_guid
  target: txn_def_guid
  type: NUMERIC(16, 0)
- source: account_no
  target: account_no
  type: VARCHAR(10)
```

```yaml
version: '1.0'
mapping_type: column
generated_at: '2025-10-08T12:10:10Z'
source:
  schema: ocn_iss
  table: txn_issuer_security
  sql: "SELECT
    "ocn_iss"."txn_issuer_security"."txn_guid",
    "ocn_iss"."txn_issuer_security"."cavv_ind",
    "ocn_iss"."txn_issuer_security"."cavv_data",
    "ocn_iss"."txn_issuer_security"."security_level_indicator"
FROM "ocn_iss"."txn_issuer_security"
WHERE EXISTS (
    SELECT 1
    FROM ocn_iss.txn_issuer_onl tio
    WHERE txn_issuer_security.txn_guid = tio.guid
)"
columns:
```

```
- source: txn_guid
  target: txn_guid
  type: NUMERIC(16, 0)
- source: cavv_ind
  target: cavv_ind
  type: NUMERIC(3, 0)
- source: cavv_data
  target: cavv_data
  type: VARCHAR(48)
- source: security_level_indicator
  target: security_level_indicator
  type: NUMERIC(3, 0)
```

13.3.10 Performans Hedefleri (KPIs)

- Düşük Bellek Ayak İzi: Veri hacminden bağımsız olarak sabit RAM kullanımı.
- Yüksek Throughput: Saniyede 100.000+ satır aktarım kapasitesi.
- Zaman Verimliliği: Milyon satırlık yüklemeleri birkaç dakika içerisine tamamlama hedefi. Aktarımlar kullanılan sistemlerin network, CPU ve bellek durumlarına göre değişiklik göstermele birlikte. Testler bu sistemler üzerinden geleneksel yöntemler ile ffengine karşılaştırılarak yapılmaktadır.

13.4 Teknik Gereksinimler ve Mimari Detaylar

13.4.1 Temel İlke

Queue-first değil, capability-first queue-aware integration.
Sistem, tek bir yöntemi tüm ortamlara dayatmak yerine her kaynak ve hedef sistemin teknik kabiliyetine göre en uygun akış modelini seçmelidir:

- Kaynakta native queue desteği varsa öncelikle onu kullan.
- Native queue yoksa ve CDC uygunsa CDC kullan.
- CDC de uygun değilse kontrollü SQL polling / outbox / staging queue table yaklaşımını kullan.
- Hedefte de aynı mantıkla en uygun yazma stratejisini seç.

Bu yaklaşım uygulanmadığında ürün teoride şık görünse bile pratikte bazı sistemlerde gereksiz karmaşık, bazı sistemlerde ise verimsiz kalır.

13.4.2 Ürün Diline Yansıması

İddia şu şekilde konumlandırılmaktadır:

- Yanlış/iddialı söylem: "Tüm DBMS'lerde queue ile aynı performansta çalışır."
- Doğru/savunulabilir söylem: "FFEngine, her kaynak ve hedef sistem için en uygun akış modelini seçerek queue-aware, low-lock veri entegrasyonu sağlar."

13.4.3 Ürün İçinde Modelleme: Üç Katmanlı Resolver

Ürün, tek bir yöntem değil, karar veren bir strateji motoru olarak modellenmelidir:

1. Source Strategy Resolver: Kaynak için native queue, CDC veya SQL polling seçimi yapar.
2. Transport Strategy Resolver: Runtime içinde in-memory micro-batch lane, broker-backed lane veya persisted buffer lane seçimini yapar.
3. Target Strategy Resolver: Hedef için native queue ingress, staging/outbox queue table, direct micro-batch writer veya bulk writer seçimini yapar.

13.4.4 Karar Kriterleri (Sadece DBMS Adına Bağlı Olmamalı)

Strateji seçimi tek başına "hangi DBMS?" sorusuna göre değil, aşağıdaki girdilere göre yapılmalıdır:

- Native queue erişimi var mı?
- CDC erişimi var mı?
- Yetki/kurulum kısıtları uygun mu?
- Hedef sistem yazma baskısına ne kadar dayanıklı?
- Gecikme (latency) beklentisi nedir?
- Throughput beklentisi nedir?
- Ordering gerekiyor mu?
- Exactly-once gerekli mi, yoksa at-least-once yeterli mi?
- Operasyonel karmaşıklık toleransı nedir?
- Kaynak sistem üretim ortamında intrusive çözüm kabul ediyor mu?

Aynı DBMS için bile müşteriye göre farklı strateji kararları çıkabilir; bu esneklik ürünün zayıflığı değil, gücüdür.

13.4.5 Capability Registry (Adapter Değil, Yetenek Kayıt Katmanı)

Sistem, "DBMS → supported capabilities → preferred strategies" şeklinde bir capability registry ile çalışmalıdır.

```
capability_registry:
  oracle:
    supported: [native_queue, cdc, sql_polling]
    preferred: [native_queue, cdc, sql_polling]
  mssql:
    supported: [native_queue, cdc, sql_polling]
    preferred: [native_queue, cdc, sql_polling]
  postgresql:
    supported: [cdc, notification, sql_polling]
    preferred: [cdc, notification, sql_polling]
  others:
    supported: [sql_polling]
    preferred: [sql_polling]
```

Burada "destekleniyor" ve "öneriliyor" alanları ayrı tutulmalıdır.

13.4.6 Strategy Score Mekanizması

Her aday yöntem için skor hesaplanmalı ve otomatik seçim/öneri mekanizması işletilmelidir:

- Verimlilik
- Güvenilirlik
- Kurulum karmaşıklığı
- Operasyon maliyeti
- Hedef uyumu
- Lock riski
- Gözlemlenebilirlik

```
strategy_selection:
  mode: auto
  priorities:
    - low_lock
    - reliability
    - throughput
  allow_fallback: true
```

13.4.7 Hard Fallback Zinciri

Kaynak ve hedef için fallback zinciri deterministik olmalıdır:

```
# source
if native_queue is available and efficient:
    use native_queue
elif cdc is available and acceptable:
    use cdc
else:
    use sql_polling

# target
if target_native_queue exists:
    use target_queue_ingress
elif staging_queue_table is supported:
    use staging_queue_table
else:
    use direct_micro_batch_writer
```

13.4.8 Neden Kritik?

Tek tip yaklaşımın oluşturacağı riskler:

- Queue desteği zayıf sistemde native queue zorlanır ve verim düşer.
- CDC güçlü olan sistemde gereksiz polling yapılır.
- Hedef bulk yazım için uygunsa gereksiz queue katmanı eklenir.
- Düşük gecikme gerekmeyen akışlarda gereksiz runtime karmaşıklığı artar.

Sonuç olarak FFEngine, "tek yöntem" ürünü değil; "en uygun yöntemi seçebilen" entegrasyon motoru olmalıdır.

13.4.9 Dialect-Tabanlı Soyutlama Katmanı (Database Agnosticism)

Sistem, veritabanına özel mantığı çekirdek koddan ayırmalıdır.

- Agnostic Driver Interface: DBAPI 2.0 standartlarına sadık kalarak, PostgreSQL, MSSQL ve Oracle için ortak bir bağlantı arayüzü sunulacaktır. Sonra

- Dynamic Dialect Mapping: Veri tipleri aşağıdaki kurallarla merkezi bir TypeMapper sınıfı üzerinden yönetilecektir : örneğin Oracle –> Postgresql
  `NUMBER with scale > 0 -> NUMERIC(p,s)`
  `NUMBER with scale = 0 -> SMALLINT/INTEGER/BIGINT by precision`
  `NUMBER without p/s -> NUMERIC`
- Veritabanına özgü SQL sentaksları (pagination, hintler, bulk insert komutları) Dialect sınıfları içinde kapsüllenecektir.
- Target DDL Generator: Kaynak veritabanından alınan metadata (column name, type, nullability), hedef veritabanının lehçesine uygun CREATE TABLE komutuna otomatik dönüştürülecektir.

13.4.10 Yüksek Performanslı Streaming I/O Mekanizması

Bellek hatalarını (OOM) önlemek için verinin tamamı asla RAM'e yüklenmez.

- Server-Side Cursors: Kaynak veritabanında `fetchmany()` kullanılarak veriler chunk'lar (ör. 10.000 satırlık paketler) halinde çekilecektir. Bu değer parametriktir.
- Binary Streaming: Mümkün olan sürücülerde (ör. psycopg3 ile COPY), veri JSON veya String yerine binary formatta akıtılarak serileştirme yükü (CPU overhead) %30 azaltılacaktır.
- Producer-Consumer Pattern: Veri okuma (Source) ve veri yazma (Target) işlemleri asenkron veya multi-threaded yapıda çalışarak I/O bekleme sürelerini minimize edecektir.

13.4.11 Bulk Yükleme Optimizasyonları (Target Specific)

Her veritabanı için en hızlı yazma metodu kullanılacaktır:

- PostgreSQL: COPY FROM STDIN veya INSERT ... VALUES (multi-row).
- MSSQL: bcp benzeri veya `fast_executemany=True` (pyodbc) optimizasyonu.
- Oracle: `executemany()` ile OCI_BATCH_ERRORS yönetimi.

13.4.12 Konfigürasyon Yönetimi (Low-Code ETL)

Geliştiricilerin Python kodu yazmasına gerek kalmadan, YAML dosyaları üzerinden tüm pipeline tanımlanacaktır.

```
# ffengine config örneği
source_db_var: ""
target_db_var: ""
```

```yaml
etl_tasks:
 - task_group_id: "source_schema_source_table_to_target_schema_target_table_name_v1"
    source_schema: "source_schema"
    source_table: "source_table_name"
    source_type: table
    target_schema: "target_schema"
    target_table: "target_table_name"
    load_method: "create_if_not_exists_or_truncate"

    where: >
      "EntryDateTime" > :dwh_oc_epr_max_entry_date_time
    order_by: ""      # push-down sıralama (opsiyonel)

    bindings:
      dwh_oc_epr_max_entry_date_time:
        from: target
        sql: |
          SELECT MAX(date_trunc ('day',"LastUpdatedDate"))
          FROM "Schema"."table_name"
        default: "1900-01-01 00:00:00"

    passthrough_full: true        # tüm tablolar için yeni mod
    passthrough_format: "binary"
    pipe_queue_max: 8             # (opsiyonel) backpressure hassasiyeti
    batch_size: 10000
    partitioning:
      enabled: true
      column: "MemberId"
      mode: "percentile"
      parts: 2
```

13.5 ETL ve Streaming Mimari Mimarisi (Deep Dive)

13.5.1 ETL & Streaming Mimarisi (High-Speed Batch)

ffengine, batch veri transferini modern streaming I/O teknikleriyle gerçekleştirir. Buradaki "Streaming", verinin kaynağından hedefe "boru hattı" (piping) mantığıyla, ara bir depolama veya devasa bellek kullanımına ihtiyaç duymadan akıtılmasıdır.

13.5.2 Veri Akış Mimarisi (Data Flow Architecture)

ffengine, geleneksel "Veriyi oku → Diske/RAM'e yaz → Hedefe gönder" döngüsünü kırarak, veriyi bir boru hattı (pipeline) içinden akan kesintisiz bir su gibi yönetir.

- Extraction (Kaynak): Server-side cursor açılarak veritabanı seviyesinde bir işaretçi (pointer) oluşturulur. Uygulama, veriyi satır satır değil, optimize edilmiş paketler (ör. 50.000 satırlık `fetchmany`) halinde çeker.
- Transformation (Dönüşüm): Veri, hedef veritabanına girmeden hemen önce "on-the-fly" (akış anında) dönüştürülür. Ağır veri işleme operasyonları (Type Casting, Naming) jeneratörler üzerinden belleğe yük bindirmeden yapılır.
- Loading (Hedef): Her dialect'in en hızlı yükleme mekanizması (Postgres COPY, Oracle OCI_BATCH) boru hattının ucuna bağlanır.

13.5.3 Teknik Bileşenler ve Protokoller

A. Streaming I/O Protokolü
Geleneksel ETL araçları veriyi JSON veya CSV olarak geçici dosyalara yazar. ffengine ise şu yolu izler:

- Binary Stream: Özellikle Postgres tarafında psycopg3 kullanarak veriyi metin tabanlı (text-based) değil, binary formatta aktarır. Bu, %30–%40 oranında daha az CPU kullanımı ve daha hızlı veri transferi sağlar.
- Backpressure (Geri Basınç) Yönetimi: Eğer hedef veritabanı yazma hızında yavaş kalırsa, Streamer modülü kaynak okuma hızını otomatik olarak dengeler (buffer throttling), böylece RAM taşmaları önlenir.

B. Dialect-Based Soyutlama (Agnostic Layer)
Mimari, veritabanına özgü davranışları çekirdekten ayırır:

- Metadata Engine: `information_schema` sorguları yerine, her DB'nin kendi native metadata API'larını kullanır (ör. Oracle için `ALL_TAB_COLUMNS`).
- Dynamic DDL: Hedef tabloda kolon yoksa, kaynak metadata bilgisiyle hedef lehçesine (dialect) uygun CREATE TABLE komutunu anlık üretir.

C. Transaction ve State Yönetimi

- Atomicity: Aktarım sırasında bir hata oluşursa, o ana kadar yüklenen paketlerin (chunks) durumuna göre "Rollback" veya "Resume from Checkpoint" stratejileri uygulanır.
- Partition-Aware Streaming: Büyük tabloları partition bazlı paralel stream'lere bölerek (Multi-threading) toplam aktarım süresini dramatik şekilde düşürür.

13.5.4 Mimarinin Bileşen Şeması

Katman	Görev	Teknoloji / Yaklaşım
Ingestion Layer	Veriyi kaynaktan çekme	Cursor-based Fetching, JDBC/ODBC Bridge
Stream Engine	Paketleme ve Buffer yönetimi	Python Generators, Memory Views
Mapping Engine	Tip dönüşümü ve DDL üretimi	Centralized Type Mapper (YAML bazlı)
Delivery Layer	Hedefe yüksek hızda yazma	Native Bulk API (COPY, BCP, OCI)
Orchestration	Görev planlama ve izleme	Apache Airflow Operator

13.6 Teknik Ekibin yukarıdakilere ek olarak Uygulama Notları

- Interface Consistency: ffengine enterprise, ffengine community sınıflarını miras almalı veya aynı "Interface"i (arayüzü) sağlamalıdır. Airflow operatörü, hangi motorun yüklü olduğunu tespit edip (automatic fallback) ona göre tetikleme yapmalıdır.
- C-Extension Güvenliği: C tarafındaki bellek sızıntıları (memory leaks) tüm Airflow worker'ını çökertebilir. Bu yüzden Enterprise versiyonunda Valgrind ile stres testleri zorunludur.
- ETL Studio UI: Community versiyonu için geliştirilen UI arayüzü, Enterprise için de kullanılacak.
- Enterprise versiyonda performans metriklerini (satır/sn) görselleştirecek şekilde genişletilmelidir.

14 Airflow Entegrasyonu ve Operasyon

Airflow'un orkestrasyon gücü, ffengine'in üretim hızıyla birleştirilir.

- ETL Studio: Airflow arayüzü içinde çalışan, DAG ve YAML'ı geliştirirken kodsuz pipeline tasarımı sunarak standartlaşmayı sağlayan eklentidir.
- Dynamic Task Mapping: Veri hacmine göre ETL studyodaki setuplara göre herbir aktarım için paralel worker task'ı oluşturulur.

15 Geliştirme Stack'i ve Standartlar

Bileşen	Community (Free)	ffengine enterprise
Geliştirme Dili	Python	C / C++ (Native ffengine enterprise core)
Bağımlılıklar	SQLAlchemy, Pandas, Airflow	Native DB Libraries (libpq, oci.h)
Hız Faktörü	1x (Referans)	5x – 10x (C-Optimization)
Dağıtım	PyPI / Docker	Binary Wheels / Private Registry
Lisans Kontrolü	Yok (Open Source)	RSA şifreli Lisans Modülü

16 Teknik Uygulama Detayları (Engineering Implementation)

Bu bölüm, ffengine'in geliştirme aşamasındaki teknik standartlarını ve modüler yapısını tanımlar.

16.1 Veri Akış Protokolü (The Streaming Protocol)

Teknik ekibin "Binary Streaming" hedefine ulaşması için izleyeceği protokol:

- Buffer Management: Bellek şişmesini önlemek için `yield` jeneratörleri kullanılacak. Veri paketleri (chunks) bellekte biriktirilmeden, bir jeneratör üzerinden doğrudan hedef yazıcıya (sink) aktarılacak.
- Type Casting Logic: Kaynak veritabanından alınan veriler, ortak bir ara formata (Internal Data Representation) çevrilmeden, doğrudan hedef veritabanının beklediği native tipe "yerinde" (in-place) veya mapping adımında dönüştürülecektir.
- Örnek: Oracle DATE tipi, hedef Postgres ise TIMESTAMP formatına string manipülasyonu olmadan, sürücü seviyesinde `datetime` objesi olarak taşınacaktır.

16.2 Dialect Soyutlama Katmanı Uygulaması

Her veritabanı desteği için bir `BaseDialect` sınıfından türetilen sınıflar yazılacaktır:

- `get_table_schema()`: Kaynaktan metadata çekme sorgusu.
- `generate_bulk_insert_query()`: Hedef veritabanının en hızlı INSERT sentaksını üretme.
- `get_data_type_map()`: Kaynaktaki X tipinin hedefteki Y tipine karşılığı.

16.3 Kullanılan Teknolojiler

- Docker: `apache/airflow:3.1.6-python3.12`
- İşletim Sistemi ve Kaynaklar: Linux (8 CPU, 32GB bellek)
- Python: `3.12`
- Apache Airflow: `3.1.6`

16.4 Altyapı ve Dağıtım (DevOps & Infrastructure)

- Dockerization: ffengine, Airflow Worker'ları üzerinde bağımsız çalışabilmesi için hafif (lightweight) bir imaj olarak paketlenecektir.

- Library Standartları:
  - PostgreSQL: psycopg3 (Native binary COPY desteği için).
  - MSSQL: pyodbc (UnixODBC üzerinden msodbcsql18 sürücüsüyle).
  - Oracle: python-oracledb (Thin mode desteğiyle, Oracle Client kurulumu gerektirmeden).
- CI/CD Pipeline: Her yeni Dialect eklendiğinde (ör. MySQL veya Snowflake), tüm mevcut veri aktarım testlerini (Integration Tests) tetikleyen bir test suit'i kurulacaktır.

16.5 Hata Yönetimi ve Gözlemlenebilirlik (Observability)

- Gelişmiş Hata İzleme: Job izleme, hata takibi ve loglama süreçleri Apache Airflow'un DAG log altyapısı üzerinden yürütülecektir.
- Standart DAG izleme ekranlarına aktarım performans metrikleri eklenecektir.
- Enterprise versiyodan, Airflow'a, DAG timeline izleme ekranı eklenecek. Böylelikle DAG'lerin ne zaman başlayıp ne zaman bittikleri tek bir ekrandan takip edilebilecek.
- Seviye Bazlı Loglama: DAG log yazımı seviye bazlı olacak; log seviyesi 1, 2 ve 3 olarak ayarlanabilecek ve sırasıyla INFO, WARNING ve ERROR mesajlarını gösterecektir.
- Hata Aşaması İzleme: Veri aktarımı sırasında hata oluşursa, hatanın meydana geldiği aşama (extract/transform/load) açık bir etiketle loglara yazılacaktır.
- Enterprise versiyodan, Real-time Metrics: Aktarım hızı (rows/sec) ve kalan tahmini süre bilgilerini Airflow loglarına basan bir ProgressTracker modülü eklenecektir.

17 İleri Düzey Teknik Detaylar (Advanced Technical Specs)

Mühendislik ekipleri, sadece mutlu yol (happy path) senaryolarını değil, sistemin limitlerini ve hata anındaki davranışlarını da bilmek ister.

17.1 Veri Tipi Uyumlaştırma ve Edge Case Yönetimi

- LOB (Large Objects) Handling: Oracle CLOB/BLOB veya Postgres BYTEA gibi büyük veri tiplerinin streaming sırasında belleği patlatmaması için bu kolonların ayrı bir "chunking" stratejisiyle mi taşınacağı yoksa belli bir boyuta kadar mı destekleneceği netleştirilmelidir.
- Precision & Scale: Finansal verilerde DECIMAL ve NUMERIC tiplerin farklı DB'ler arasındaki hassasiyet farklarının (ör. Oracle'ın 38 basamak desteği vs. Postgres) nasıl handle edileceği TypeMapper içinde tanımlanmalıdır.

17.2 Paralelizasyon Stratejisi

- Vertical vs Horizontal Scaling: Vertical: Tek bir ffengine instance'ı içinde multiprocessing kullanarak farklı partition'ları paralel çekmek.
- Horizontal: Airflow'un dinamik task mapping (Dynamic Task Mapping) özelliğini kullanarak, tabloyu parçalara bölüp her parçayı farklı bir Airflow Worker'a dağıtmak.

17.3 İzlenebilirlik (Logging & Telemetry)

- Performance Profiling: Her aktarımın sonunda; "Saniye başına satır", "I/O bekleme süresi", "CPU kullanımı" gibi metriklerin bir `summary_report` olarak üretilmesi.

18 Teknik Karar Kaydı (ADR — Architecture Decision Records)

- Neden Python? Ekosistemin genişliği, Airflow uyumu ve psycopg3/oracledb gibi kütüphanelerin yüksek performansı nedeniyle.
- Neden YAML? Okunabilirlik, versiyon kontrolü (Git) dostu olması ve teknik olmayan ekiplerin de (Data Analyst) konfigürasyon yapabilmesi için.

19 Gereksinimler

19.1 Fonksiyonel Olmayan

- Ölçeklenebilir depolama
- Güvenli erişim ve denetim kayıtları

20 Başarı Ölçütleri

- Aktarım hızı (Throughput: Rows/Second)
- Veri tutarlılık ve data doğruluğu
- Büyük veri setlerinde bellek kullanımının sabit kalması
- Pipeline kurulum süresindeki kısalma (saatlerden dakikalara)

21 Pazar Araştırması: Pazar Konumlandırması ve Hedef Pazar Payı

FFEngine, genel veri entegrasyonu pazarının tamamını hedefleyen yatay bir ürün olarak değil; Airflow-native mimari, veritabanları arası yüksek performanslı veri taşıma, düşük kilit etkili yazım ve queue-aware çalışma yaklaşımına odaklanan uzmanlaşmış bir veri entegrasyon platformu olarak konumlandırılmaktadır. Bu yaklaşım, ürünü klasik batch ETL araçları ile daha karmaşık ve operasyonel maliyeti yüksek gerçek zamanlı entegrasyon çözümleri arasında farklılaştırılmış bir konuma yerleştirmektedir.

Küresel veri entegrasyonu pazarının 2025 yılı itibarıyla yaklaşık 17.6 milyar ABD doları seviyesinde olduğu ve 2030 yılına kadar yaklaşık 33.2 milyar ABD dolarına ulaşmasının beklendiği değerlendirilmektedir. Buna ek olarak, veri pipeline araçları pazarının da hızlı büyüyen bir alt segment olduğu ve 2024 yılı itibarıyla yaklaşık 12.1 milyar ABD doları seviyesinde bulunduğu öngörülmektedir. Bu veriler, FFEngine'in yalnızca klasik ETL bütçelerinden değil, modern veri hattı ve operasyonel veri hareketi yatırımlarından da pay alabilecek bir ürün olduğunu göstermektedir.

Pazarın mevcut yapısı dikkate alındığında, FFEngine için en doğru strateji, toplam veri entegrasyonu pazarının tamamına aynı anda hitap etmek yerine; yüksek hacimli veritabanları arası veri taşıma, kontrollü mikro-batch yazım, düşük kilitlenme riski, operasyonel güvenilirlik ve Airflow tabanlı orkestrasyon ihtiyacı bulunan daha dar fakat yüksek değerli bir alt segmente odaklanmaktır. Bu çerçevede, FFEngine'in başlangıç aşamasında toplam pazarın yaklaşık %1 ila %3'üne karşılık gelen, uzmanlaşmış ve yüksek katma değerli bir hizmet verilebilir pazarı hedeflemesi uygun görülmektedir. Bu oran, ürünün teknik farklılaştırıcıları ve hedef kullanım senaryoları dikkate alınarak oluşturulmuş stratejik bir pazar segmentasyonu varsayımıdır.

Bu hedef segment içinde değerlendirildiğinde, FFEngine için pazar payı beklentisi üç seviyede ele alınabilir. Muhafazakâr senaryoda, ürünün hedef segment içinde %0.5 ila %1.0 seviyesinde pay alması; baz senaryoda %1.5 ila %3.0 aralığına ulaşması; güçlü bir ürün-pazar uyumu, ekosistem yayılımı ve referans müşteri kazanımı durumunda ise %4 ila %7 bandına ilerlemesi mümkün görülmektedir. Daha yüksek oranlar uzun vadeli kategori liderliği senaryosu kapsamında teorik olarak mümkün olmakla birlikte, mevcut aşamada temel planlama varsayımı olarak değerlendirilmemelidir.

FFEngine Community sürümü açısından temel amaç doğrudan gelir payı yaratmaktan çok, teknik kullanıcı tabanı oluşturmak, ürün yayılımını hızlandırmak ve Enterprise sürüm için doğal bir geçiş kanalı oluşturmaktır. Enterprise sürüm ise özellikle operasyonel güvenilirlik, kontrollü teslimat, queue-aware çalışma modeli, düşük kilit etkili veri yazımı ve büyük ölçekli veri hareketi gerektiren kurumsal senaryolarda gelir üreten ana katman olarak konumlandırılmaktadır. Airflow ekosisteminin geniş kurumsal kullanım tabanı dikkate alındığında, bu iki katmanlı yaklaşımın hem teknik benimsenmeyi hem de ticari ölçeklenebilirliği desteklemesi beklenmektedir.

Bu doğrultuda, FFEngine için önerilen stratejik pazar payı hedefleri aşağıdaki şekilde tanımlanabilir:

- İlk üç yıllık dönemde, hedef segment içinde %1.5 ila %3.0 seviyesinde pay elde edilmesi,
- Orta vadede %4 ila %5 bandına ulaşılması,
- Uzun vadede uygun ürün-pazar uyumu ile ekosistem desteği sağlandığı takdirde %7 ve üzeri segment payının hedeflenmesi.

Bu hedefler, toplam veri entegrasyonu pazarına değil, FFEngine'in uzmanlaşmış hedef alt segmentine ilişkindir ve bu şekilde yorumlanmalıdır.

22 Riskler ve Varsayımlar

- Kullanıcıların mevcut araçlarından geçiş maliyeti
- Kurumsal güvenlik ve uyumluluk beklentileri
- Ürün kapsamının erken aşamada fazla genişlemesi

23 Yol Haritası

23.1 Kısa Vadeli (0–3 ay)

- Community versiyonun github üzerinden kolaylıkla kurulumu
- Erken kullanıcılarla pilot kullanım

23.2 Orta Vadeli (3–6 ay)

- Community'nin yönetilmesi, yeni özellliklerin devreye alınması
- Enterprise versiyon Engine'inin geliştirme sürecinin tamamlanması

23.3 Uzun Vadeli (6+ ay)

- Enterprise cersiyonlar ile POS çalışmamaları
- Community ile birlikte çalışma toplantıları, FFEngine tanıtım çalışmaları

24 Stratejik Yol Haritası (2026–2027)

24.1 Faz 1: Core & Community Launch (Q1 2026)

- ffengine community Release:UI, Python çekirdeği ile standart aktarım metodlarının (Postgres, MSSQL, Oracle vb) yayına alınması.
- Web Sitesi: ffengine.com üzerinden "Community" dökümantasyonunun ve kurulum rehberlerinin yayını.

24.2 Faz 2: ffengine enterprise Core Development (Q1 2026)

- ffengine enterprise Core Development: Performansın darboğaz olduğu noktalarda Python yerine C kodlarıyla yazılmış veri taşıma motorunun geliştirilmesi.
- Binary Stream Optimization: C seviyesinde bellek yönetimi (Buffer allocation) ile veriyi ham formatta taşıyacak mimarinin kurulması.
- Enterprise Pilot: Seçili tasarım ortaklarıyla yüksek hacimli aktarım testleri.

24.3 Faz 3: Performance & Scalability (Q2 2026)

- Advanced ETL Studio: Enterprise kullanıcıları için görsel şema eşleştirme ve performans analiz araçları.

24.4 Faz 3: Market Expansion (2027)

- Cross-DB Cloud Connectors: Enterprise versiyonuna özel Snowflake ve BigQuery C-Konnektörleri.
- Hybrid Cloud Streaming: On-premise sistemler ile Cloud veritabanlarının aktarım paketleri.

25 FFEngine Feature Matrix (Community vs Enterprise)

Net ilke: Community tarafı ağırlıklı olarak *query-centric ETL* yaklaşımıyla veri taşır. Enterprise tarafı *queue-aware data movement* yaklaşımıyla akışı yönetir, kontrol eder ve teslimat garantisini güçlendirir.

25.1 1) Execution Model & Runtime

Özellik	Community	Enterprise
Execution modeli	Query-centric (SELECT → batch write)	Queue-aware (producer → ingress → egress → consumer)
Streaming	Temel (fetchmany / COPY)	Gelişmiş (stream + queue semantics birlikte)
Pipeline tipi	Batch / micro-batch	Continuous + micro-batch (flow-controlled)
Runtime	Python (DB-API)	Native/C + queue runtime + workers
Lane (paralel akış)	Sabit, basit	Dinamik, adaptive, multi-lane

25.2 2) Source (Okuma)

Özellik	Community	Enterprise
SELECT polling	Var	Var
Partitioned read	Var (manuel)	Var (manuel + otomatik/uyarlamalı)
Server-side cursor / streaming	Var	Var (optimize)
Native DB queue (AQ/Service Broker)	Yok	Var
Outbox/queue-table tüketim	Sınırlı (manuel)	Var (standart adapter)
Capability-based source seçimi	Basit	Gelişmiş (auto + skor)

25.3 3) Target (Yazma)

Özellik	Community	Enterprise
Direct write (insert/upsert)	Var	Var
Micro-batch write	Var	Var (adaptive)
Native bulk API (COPY/bcp/SqlBulkCopy)	Sınırlı/temel	Var (optimize/zero-copy)
Target ingress queue	Yok	Var
Controlled writer (küçük commit, throttle)	Yok	Var
Backpressure (hedefe göre yavaşlatma)	Yok	Var

25.4 4) Delivery & Reliability (Teslimat Semantiği)

Özellik	Community	Enterprise
Delivery modeli	Best-effort	At-least-once (+ idempotency ile pratik exactly-once)
Ack / Nack	Yok	Var
Message-level retry	Yok (task-level var)	Var (policy + backoff)
DLQ (Dead Letter Queue)	Yok	Var
Checkpoint / Offset store	Basit (watermark)	Gelişmiş (offset/ack tabanlı)
Resume (kaldığı yerden devam)	Sınırlı	Güçlü
Idempotency (duplicate kontrolü)	Yok	Var
Ordering (partition/key bazlı)	Yok	Var

25.5 5) Partitioning & Parallelism

Özellik	Community	Enterprise
Basic partitioning (range/hash)	Var (manuel)	Var
Parallel read/write	Var	Var
Auto partitioning	Yok	Var
Adaptive partitioning	Yok	Var
Skew detection / düzeltme	Yok	Var
Queue-aware partition (key-based routing)	Yok	Var
Dynamic lane scaling	Yok	Var

25.6 6) Transformation

Özellik	Community	Enterprise
Basic transform (column mapping)	Var	Var
SQL-based transform	Var	Var
In-stream transform	Sınırlı	Var
Event/message-based transform	Yok	Var
Schema evolution handling	Temel	Gelişmiş

25.7 7) Configuration & Orchestration

Özellik	Community	Enterprise
YAML config	Var	Var (aynı şema)
Auto-DAG (Airflow)	Var	Var
Dynamic Task Mapping	Var	Var (daha yoğun kullanım)
Pools / concurrency control	Var	Var
Strategy selection (source/target)	Basit	Gelişmiş (auto + fallback)
Capability discovery	Yok	Var

25.8 8) Monitoring & Observability

Özellik	Community	Enterprise
Basic logging	Var	Var (structured + zengin)
Metrics (row/sec, duration)	Temel	Gelişmiş (per-lane, queue depth, lag)
Pipeline visibility	Sınırlı	End-to-end (source → queue → target)
Error tracking	Temel	Detaylı (DLQ, retry history)

25.9 9) Performance

Özellik	Community	Enterprise
Throughput	Orta–yüksek	Çok yüksek (native + queue + paralel)
Latency	Batch odaklı	Düşük + kontrol edilebilir
Memory kullanımı	Düşük	Düşük–orta (ama kontrollü)
Büyük veri stabilitesi	Sınırlı	Yüksek (flow control)

25.10 10) DBMS Capability Kullanımı

Özellik	Community	Enterprise
DB-agnostic çalışma	Var	Var
Native queue kullanımı	Yok	Var
Queue emulation (outbox/staging)	Sınırlı	Var
DB-specific optimizasyon	Temel	Gelişmiş

25.11 Net Ürün Ayrımı (Tek Bakışta)

Kriter	Community	Enterprise
Felsefe	Query-centric ETL	Queue-aware data movement
Güçlü olduğu yer	Basitlik, hızlı kurulum	Güvenilirlik, kontrol, ölçek
Risk yönetimi	Düşük	Yüksek
Hedef kullanım	Standart veri taşıma	Kritik / büyük / hassas pipeline'lar

25.12 En Kritik Fark (Ürün Mesajı)

Community veri taşır.
Enterprise veri akışını yönetir, kontrol eder ve garanti eder.

25.13 Enterprise Execution Strategy

Bu bölüm, ffengine enterprise çalışma prensibini netleştirir: ürün tek bir fiziksel aktarım yöntemini dayatmaz; bunun yerine kaynak/hedef çiftinin kapasitesine göre en uygun stratejiyi seçer.

25.13.1 Execution Model

ffengine enterprise, yürütmeyi *capability-aware* bir modelle ele alır. Çekirdek motor, kaynak ve hedefin sürücü/DBMS özelliklerini (bulk API desteği, transaction davranışı, concurrency limiti, akış penceresi, retry maliyeti) dikkate alarak runtime akışını belirler.

25.13.2 Capability-Driven Strategy Selection

Strateji seçimi tekil bir kalıpla değil, yetenek matrisiyle yapılır:

- Kaynak tarafı güçlü streaming ve hedef tarafı bulk ingest destekliyorsa queue-aware boru hattı tercih edilir.
- Kaynak/hedef yetenekleri sınırlıysa query-centric veya hibrit bir yol seçilir.
- Seçim kriterleri; throughput, gecikme, bellek izi, idempotency ihtiyacı ve operasyonel kontrol gereksinimidir.

25.13.3 Queue-Aware vs Query-Centric

- ffengine community için varsayılan yaklaşım query-centric ETL'dir.
- ffengine enterprise için varsayılan yaklaşım queue-aware controlled movement'tır.
- Ancak enterprise katmanı da gerektiğinde query-centric veya hibrit modda çalışabilir; ürün tek yöntemi dayatmaz.
- Tüm DBMS'lerde aynı fiziksel queue yapısı beklenmez; kuyruk modeli, ilgili sürücünün ve veritabanının sunduğu yeteneklere göre uyarlanır.

26 Kapsam Dışı Konular

- Veri Kalitesi ve Validasyon (Data Quality): Akış anında veri tiplerinin doğruluğu dışında, iş kurallarına dayalı (ör. null kontrolü, regex eşleşmesi) kontrol mekanizması.
- Veri Maskeleme ve Anonimleştirme (Data Masking): Finansal verilerle çalışırken KVKK/GDPR uyumu için hassas kolonların akış anında maskelenmesi.
- Change Data Capture (CDC): FFEngine enterprise bir CDC aracı değildir; veritabanı loglarından (WAL/Redo log) gerçek zamanlı değişim yakalama yeteneği kapsam dışıdır. Enterprise sürümü Kafka veya RapidMQ'yu kaynak ya da hedef olarak kullanır ve bu sistemler arasındaki veri hareketini queue-aware şekilde yönetir.